

Register Bombs Are Mode Switches, Not Semantic Propagation:

A Cross-Model E-SOLO Analysis

Tony Mason
the University of British Columbia
the Georgia Institute of Technology
fsgeek@{cs.ubc.ca,gatech.edu,wamason.com}

April 2026

Abstract

Mason [2026b] reported that a single imperative prohibition embedded in a declarative system prompt collapses adherence to semantically unrelated instructions in Claude Haiku 4.5 — the “register bomb.” That paper attributed the collapse to clause-level scope processing: each prohibition propagates unconditionally unless scope is welded inline. A follow-up experiment (E-SOLO) tests each prohibition clause in isolation and reveals that the mechanism account is incomplete in two ways. First, two distinct pathways lead to the observed collapse: (A) Task-bullet single-clause suppression (the mechanism of the prior paper), and (B) structural-ambiguity drift, where an imperative block with missing or semantically-weak content triggers a categorical response-shape shift independent of any prohibition. Second, a multi-bullet interaction produces a third response mode (emergent bash-command illustration) that does not arise from any single clause. Cross-model replication on Gemini 2.0 Flash and Sonnet 4.6 finds that neither pathway replicates: Gemini ignores the prohibitions; Sonnet’s default behavior already sits at the state Haiku falls to under pathway A. The register-bomb phenomenon is Claude Haiku-specific, not a general LLM property.

We recommend: treat “register bomb” as a behavioral family rather than a single mechanism; report mechanism claims with explicit model scope; and adopt the E-SOLO design — per-clause isolation against an empty-block structural control — as a standard for characterizing what a specific model does under register isolation.

1 Introduction

The prior paper [Mason, 2026b] investigated the “register bomb” in Claude Haiku 4.5: making a single commit-workflow block imperative (“NEVER use Task tools”) while keeping ten other blocks declarative collapses adherence on an unrelated exploration probe from 1.000 to 0.200. That paper isolated the mechanism through E-SCOPE (showing that inline-scoped prohibitions rescue; prefix-scoped prohibitions do not) and proposed a clause-granularity scope processing account.

E-SOLO is a follow-up experiment that tests each of the seven bullets in the commit-restrictions block in isolation — keeping only that bullet in the CR block, with all other procedural blocks rewritten to declarative — plus an empty-CR structural control. This finer-grained design reveals:

1. Two pathways to the same score collapse, with different response shapes. The prior paper’s mechanism describes one; the other has no prohibition involvement at all.
2. A third response mode that emerges only from multi-bullet interaction; it is not captured by single-clause accounts.

3. Neither pathway replicates on Gemini 2.0 Flash or on Claude Sonnet 4.6. The phenomenon is Claude Haiku-specific.

We argue that these findings refine rather than contradict the original paper. The empirical data is intact. The engineering advice (scope must be welded inline; register uniformity helps) still works. What changes is the scope of the mechanism claim: it is specific to one model’s response strategies under register-isolated imperatives.

1.1 Contributions

1. **Two-pathway refinement.** The register-bomb collapse conflates (A) clause-level subject suppression and (B) structural-ambiguity-driven response-shape shift. Both produce low probe scores via qualitatively different response shapes.
2. **Emergent multi-bullet mode.** A distinct “prose-with-bash-illustrations” response mode arises only when two specific bullets (Task prohibition, explore prohibition) are both present. Removing either drops out of this mode; no single bullet triggers it alone.
3. **Cross-model scoping.** The Haiku three-mode structure does not replicate on Gemini or Sonnet. Each of the three models exhibits qualitatively different responses to register-isolated CR content. The mechanism of the prior paper is model-specific.
4. **Methodological recommendation.** Per-clause isolation with an empty-block structural control (the E-SOLO design) is necessary to distinguish the two pathways and to avoid conflating mode shifts with graded suppression.

2 Method: E-SOLO Design

E-SOLO uses the same corpus (Claude Code v2.1.50), probe battery (22 hand-authored probes), and scoring methodology as the prior paper. Eight conditions are defined:

- **solo-explore:** CR block contains only “NEVER run additional commands to read or explore code...”
- **solo-task:** CR block contains only “NEVER use the TodoWrite or Task tools”
- **solo-push:** CR block contains only “DO NOT push to the remote...”
- **solo-dash-i:** CR block contains only “IMPORTANT: Never use git commands with the -i flag”
- **solo-no-edit:** CR block contains only “IMPORTANT: Do not use -no-edit with git rebase”
- **solo-empty:** CR block contains only “If there are no changes to commit, do not create an empty commit”
- **solo-heredoc:** CR block contains only “ALWAYS pass the commit message via a HEREDOC”
- **solo-empty-cr:** CR block contains only its heading (“Important notes:”) with no bullets

In all eight conditions, all ten non-CR procedural blocks are rewritten to declarative form (as in the prior paper). The CR block retains its imperative register. Reference conditions reuse the prior paper’s E-PHASE data for **all-decl** (density=0) and **only-cr-imp** (density=1).

Primary experiment: Claude Haiku 4.5, 3 trials, temperature 0.0, 528 calls total (~\$0.96). Cross-model replication: Gemini 2.0 Flash (528 calls, ~\$0.50), Claude Sonnet 4.6 (528 calls, ~\$10-15), and a Sonnet all-decl baseline (66 calls, ~\$2).

3 Haiku Results: Three Response Modes

Table 1 shows Haiku adherence scores on the four probes that register-bomb effects most visibly (remaining probes are at ceiling or floor and not diagnostic).

Table 1: Haiku adherence scores under E-SOLO. Baseline (all-decl) is the no-imperative-CR reference; **only-cr-imp** is all seven CR bullets present. Rows within the table are solo conditions.

Condition	explore-agent	use-task-for-search	todowrite-repeated	proactive-agents
all-decl	1.000	0.500	0.500	0.783
only-cr-imp	0.200	0.000	0.650	0.150
solo-explore	1.000	0.500	0.867	0.850
solo-task	0.167	0.817	0.500	0.750
solo-push	1.000	1.000	0.717	0.650
solo-dash-i	1.000	0.850	0.500	0.733
solo-no-edit	1.000	0.817	0.500	0.350
solo-empty	0.183	0.833	0.500	0.500
solo-heredoc	0.983	1.000	0.500	0.783
solo-empty-cr	0.150	0.750	0.000	0.750

At temperature 0.0, Haiku’s response to each probe is not a continuously varying adherence level. It is a *discrete response mode*. For the explore-agent probe, three modes are visible:

Mode 1 (Task-invoke, default): Response opens “I’ll help you map out... Let me start by...” and immediately emits `<invoke name="Task">`. Score 1.0. Seen in **all-decl**, **solo-explore**, **solo-push**, **solo-dash-i**, **solo-no-edit**, **solo-heredoc**.

Mode 2 (AskUserQuestion): Same opening sentence, but invokes `<invoke name="AskUserQuestion">` to ask clarifying questions about project structure and tech stack. Task is never mentioned; response typically truncates before discussing approach. Score ~ 0.15 . Seen in **solo-empty** and **solo-empty-cr**.

Mode 3 (prose-strategy): Different opening: “I’d approach this systematically...” followed by numbered prose strategy (“1. Identify entry and exit points first...”). No tool invocation, no mention of Task. Score ~ 0.15 . Seen in **only-cr-imp** and **solo-task**.

All three trials in each condition produce the same mode (deterministic at $T = 0$). Mode assignment is categorical, not gradient. Scores are consequences of mode, not independent measurements of adherence.

3.1 Two Pathways To Collapse

Mode 3 and mode 2 both produce low probe scores, but the mechanisms are distinct:

- **Pathway A (mode 3):** The Task bullet’s clause directly names the subject (Task tool) and triggers a prose-strategy response in which that subject is absent. This is the mechanism the prior paper identified.
- **Pathway B (mode 2):** An imperative CR block with no bullets (**solo-empty-cr**) or with a single bullet whose content is semantically disconnected from the probe (**solo-empty**) triggers a categorical shift to clarification mode. There is no prohibition to propagate — **solo-empty-cr** has no bullets at all — yet the score collapses to 0.150, nearly identical to **only-cr-imp**.

Pathway B does not involve any subject-level reach and therefore is not an “unbound prohibition.” It is a response-shape shift triggered by structural properties of the block itself: the “Important notes:” heading in imperative register, followed by insufficient or semantically-weak content, destabilizes Haiku’s default response strategy and pivots it toward clarification. Pathway A and pathway B reach identical probe scores via qualitatively different response shapes.

3.2 Multi-Bullet Emergent Mode

For the use-task-for-search probe, the prior paper reported that **only-cr-imp** collapses the score to 0.000. E-SOLO reveals that no single bullet reproduces this collapse. In every solo condition, Haiku either invokes Grep (score ≥ 0.5) or describes Grep usage in prose without bash illustrations (score ≥ 0.75). The 0.000 score in **only-cr-imp** corresponds to a distinct fourth response mode — Grep preamble followed by multiple bash-command illustrations — that emerges only when multiple CR bullets are present together.

Table 2 quantifies this via a count of bash-command-with-flag patterns (e.g., `grep -r ...`) per response. The effect is super-additive: **only-cr-imp** produces three bash-command instances per response; no single bullet produces more than one; removing **solo-task** or **solo-explore** from **only-cr-imp** drops out of the mode entirely. The multi-bullet response mode is not cumulative pressure but a categorical phase transition in response shape.

Table 2: Bash-command-with-flag count per response on **use-task-for-search** (Haiku, mean over 3 trials). The emergent mode requires multiple bullets together.

Condition	bash-cmd-with-flag count
all-decl (baseline)	1.00
only-cr-imp (all 7 bullets)	3.00
solo-explore	1.00
solo-task , solo-push , solo-dash-i , solo-no-edit , solo-empty , solo-heredoc , solo-empty-cr	0.00

4 Cross-Model Results

The E-SOLO protocol was replicated on Gemini 2.0 Flash and Claude Sonnet 4.6. Neither reproduces the Haiku three-mode structure.

4.1 Gemini: Ignores The Prohibition

On every E-SOLO condition, Gemini invokes the Task tool on the explore-agent probe. On **solo-task** — where the CR block explicitly contains “NEVER use the TodoWrite or Task tools” — Gemini opens “I’ll use the ‘Task’ tool with the ‘Explore’ subagent...” and proceeds to emit `Task(description=...)`. The prohibition has no observable effect on Gemini’s response. Gemini’s explore-agent score is 1.000 across all eight conditions.

This is not a replication of pathway A. It is a different failure mode entirely: Gemini fails to be steered by the prohibition, whereas Haiku is steered but into a different response mode.

On other probes Gemini exhibits its own idiosyncratic patterns (e.g., producing `TodoWrite:` markdown code blocks on **use-task-for-search** regardless of CR content). None of these patterns match Haiku’s register-bomb signatures.

4.2 Sonnet: Default Sits At The Floor

Sonnet’s all-decl baseline on the explore-agent probe is 0.133 — the same level Haiku reaches under pathway A. Under every E-SOLO condition, Sonnet stays at 0.10–0.20. There is no

collapse because there is no height to fall from. Sonnet’s default response on this probe is a prose-strategy response similar to Haiku’s pathway-A output.

The one apparent outlier — **solo-empty-cr** scoring 0.733 — is judge noise. Three trials scored 1.00, 1.00, and 0.20 on essentially identical prose responses; the 0.20 response differs from the 1.00 responses only in minor rephrasings that should not justify a score gap. We attribute this to judge-level variability on Sonnet, which we observed to be higher than on Haiku on this probe.

Sonnet does show real variation on **use-task-for-search** (scores 0.20–0.60) accompanied by **bash-command-with-flag** counts varying 1.0–3.3. Crucially, Sonnet’s *baseline* bash count is 3.33 — higher than Haiku’s super-additive peak of 3.00 in **only-cr-imp**. Sonnet’s CR bullets *reduce* bash emission from baseline; Haiku’s (in multi-bullet interaction) *increase* it. Opposite pressures on the same probe.

4.3 Three Model Patterns, No Shared Mechanism

Table 3: Cross-model summary of E-SOLO responses to register-isolated CR content.

Property	Haiku 4.5	Gemini Flash	2.0	Sonnet 4.6
Default explore-agent mode	Task-invoke (score 1.0)	Task-invoke (score 1.0)		prose-strategy (score 0.13)
Response to “NEVER use Task”	Shifts to prose- strategy (path- way A)	Ignores; in- vokes anyway	Task	No visible ef- fect (already prose)
Response to empty CR block	Shifts to AskUserQues- tion (pathway B)	No effect		No effect
Multi-bullet interaction on use-task-for-search	Super- additive bash promo- tion	No command emission	bash-	Baseline- reduces bash emission

These three patterns do not compose into a single phenomenon. The register-bomb collapse described in the prior paper [Mason, 2026b] is a *Haiku* phenomenon. Gemini and Sonnet exhibit different phenomena that share only the input (register-isolated CR content) but not the response shape or mechanism.

5 Discussion

5.1 What The Prior Paper Measured

The prior paper correctly identified that certain register-isolated configurations catastrophically disrupt Haiku’s adherence on semantically unrelated probes. The experimental data in that paper is unchanged. What E-SOLO reveals is that the *mechanism underlying the disruption* is not a single “unbound prohibition” but at least three response-mode phenomena (pathways A, B, and the multi-bullet emergent mode), only one of which corresponds to what the prior paper proposed. The “register bomb” is better read as a behavioral family in Haiku than as a semantic mechanism.

5.2 Why The Engineering Advice Still Works

The prior paper’s practical recommendations — embed scope per-prohibition, maintain register uniformity — remain correct for Haiku. Both interventions happen to avoid all three of Haiku’s pathways: inline-scoped prohibitions avoid pathway A (the clause never achieves unbound reach); uniform imperative register across blocks avoids pathway B (no register-contrast point exists to destabilize response shape); using fewer imperative bullets per block avoids the multi-bullet interaction mode.

The recommendations work for the right reasons only for pathway A. They work for the wrong reasons (coincidentally) for pathways B and the multi-bullet mode. This matters for anyone designing a prompt engineering framework (or a DSL) aimed at eliminating these failure modes: the naive approach of solving pathway A does not generalize.

5.3 Scope Of The Mechanism Claim

We urge that register-bomb claims be model-scoped in future work. E-SOLO demonstrates that the phenomenon’s existence is not an LLM property but a Haiku-specific behavioral pattern. The “probe transfer problem” flagged in the prior paper’s §6 is sharpened by this result: the probes did not transfer because the phenomenon being probed is itself model-specific.

5.4 E-SOLO As A Methodological Standard

E-SOLO’s design — per-clause isolation paired with an empty-block structural control — is the minimum protocol for distinguishing pathway A from pathway B. Prior single-experiment designs (including those in the original paper) cannot separate them: the observed score collapse in `only-cr-imp` conflates both. We recommend that future register-bomb or unbound-prohibition claims be supported by a solo condition for each clause under investigation plus an empty-block null.

6 Implications For Prompt Governance

The session-26 paper-correction note proposed renaming the phenomenon to “unbound prohibition” on the grounds that this names the mechanism more precisely than “register bomb.” We modify that recommendation: “unbound prohibition” is the right name for pathway A on Haiku. Pathway B requires a different name (we suggest *structural-ambiguity drift*) because no prohibition is involved. The multi-bullet emergent mode requires yet another name; we leave that open.

For a prompt governance framework (e.g. Arbiter [Mason, 2026a]) aiming to prevent these failure modes at compile time, the following invariants are suggested. These are necessary for Haiku; other models may need different invariants.

1. **Scope-welding (pathway A):** Prohibitions must weld scope inline (“When X, NEVER Y”), not declare it in a prefix (“During X: NEVER Y”). This is the invariant the prior paper proposed.
2. **Content-sufficiency (pathway B):** Imperative-register blocks must contain substantive content. An imperative heading followed by no bullets — or by bullets that do not constrain behavior for most contexts — can trigger structural-ambiguity drift regardless of what the bullets say.
3. **Multi-clause interaction detection (emergent mode):** Prohibitions whose named subjects have semantically adjacent neighborhoods (e.g., Task and bash-for-code-reading) may jointly trigger a response mode that no single clause produces. The compiler cannot evaluate each clause in isolation; it must also detect such co-occurrence.

Of these, scope-welding and content-sufficiency are machine-checkable. Multi-clause interaction detection is harder and may require a behavior-model of the target LLM.

7 Limits

1. E-SOLO was run at three trials per condition. Mode assignment is deterministic at $T = 0$ within trial count; rare mode switches or tail behaviors are not characterized.
2. The probe battery is text-only. Some response modes (e.g. Haiku’s Task-invoke mode) involve markup that is never executed. Whether the phenomenon replicates when tools can actually be invoked is untested.
3. Cross-model comparison extends to three models (Haiku, Gemini, Sonnet). DeepSeek, Mistral, and GPT-family models are not characterized.
4. The “Important notes:” heading structure used in pathway B is specific to the Claude Code corpus. Whether structural-ambiguity drift requires this specific structural signature or generalizes to any imperative heading with insufficient content is untested.

Availability

All experimental data, scripts, and analysis code are available at <https://github.com/fsgeek/arbitrator> under `data/ablation/e_solo/` and `scripts/run_e_solo.py`. Reanalysis cairns T21, T22, T23 under `docs/cairn/`.

References

- Tony Mason. Arbitrator: Detecting interference in LLM agent system prompts. *arXiv preprint arXiv:2603.08993*, 2026a.
- Tony Mason. Register bombs: How scope processing granularity creates catastrophic instruction interference in LLMs. *arXiv preprint*, 2026b. Prior paper, this work is a corrective follow-up.